

DEVELOPER HANDOVER DOCUMENT

McKinsey & Company AI Research Engine

Complete file-by-file working reference for the React frontend, FastAPI application, Gemini/Tavily AI pipeline, and Supabase data layer.

Repository: [mckinsey-ai-market-research-engine](#)

Prepared from the current source tree • 23 September 2026

1. What the application does

This is an authenticated market-research application. A signed-in user enters a natural-language business question. The backend creates a research job, uses Gemini to break it into research tasks, uses Tavily to find and extract web content, asks Gemini to extract and validate evidence, generates a cited report, stores the artifacts in Supabase, and returns the completed report to React.

```
User → React Dashboard → POST /api/research/ → FastAPI
    → ResearchService → ResearchPipeline
    → Planner → Tavily search → Extraction → Validation → Citations → Report → Linker
    → Supabase persistence → React ReportView
```

Important runtime fact: the pipeline runs synchronously inside the POST request. `ResearchProgress.jsx` provides a timed visual animation while the browser waits; it does not receive true per-agent progress from the server.

2. Layer map

Layer	Primary location	Responsibility
Presentation	<code>frontened/src/</code>	User interface, Supabase browser session, navigation, API requests, report rendering.
Application	<code>backened/backend/</code>	HTTP routes, JWT verification, ownership checks, orchestration, database repositories, errors and middleware.
Intelligence / data	<code>backened/ai/</code> and <code>backened/backend/db/</code>	LLM agents, Tavily integration, typed schemas, pipeline rules, Supabase database schema.

3. Presentation layer — frontend files

All browser application code is under `frontened/src/`. Vite starts from `main.jsx`, React renders `App.jsx`, and the route determines which page component is shown.

`frontened/src/main.jsx`
Purpose: Browser entry point.

- Mounts the React application into the DOM element created by `frontened/index.html`.
- Imports global CSS, then renders `<App />`.
- Normally this file contains no product logic; it is the startup bridge between Vite and React.

`frontened/src/App.jsx`

Purpose: Root component and route map.

- Wraps the app in `BrowserRouter`, `ThemeProvider`, and `AuthProvider`.
- Defines public routes: `/login` and `/signup`.
- Wraps private routes with `ProtectedRoute`: `dashboard`, `research`, `history`, `reports`, `sources`, `settings`, `methodology`, and `about` pages.
- Routes unmatched paths back to `/`.

`frontened/src/index.css`

Purpose: Global CSS and visual tokens.

- Imports/configures Tailwind CSS.
- Defines common colours, typography, animation helpers, and light/dark design values used by class names throughout the UI.

Frontend API and Supabase integration

`frontened/src/lib/supabaseClient.js`

Purpose: Create the browser-side Supabase client.

- Reads `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`.
- Calls `createClient(...)` from `@supabase/supabase-js`.
- Enables persisted sessions, automatic token refresh, and URL-based auth callback detection.
- Logs a clear developer error if required frontend credentials are placeholders or missing.
- **Security:** it must use the public anon key only, never the backend service key.

`frontened/src/api/client.js`

Purpose: Single HTTP client for FastAPI.

- `authHeaders()` reads the current Supabase session and returns an `Authorization: Bearer ...` header.
- `request(path, options)` adds JSON and auth headers, calls `fetch`, parses JSON/text responses, and converts failures into `ApiError`.
- `ApiError` preserves HTTP status and server error detail for pages to display.
- The exported `api` object exposes `listResearch`, `createResearch`, `getJob`, `getTasks`, `getSources`, `getEvidence`, `getValidations`, and `getReport`.

Frontend state and reusable components

`frontened/src/context/AuthContext.jsx`

Purpose: Share Supabase authentication state across React.

- `AuthProvider` loads the initial session via `supabase.auth.getSession()`.
- Subscribes to `onAuthStateChange`, so login/logout/token changes update all consumers.
- Exposes `session`, `user`, `loading`, `signUp`, `signIn`, and `signOut`.
- `useAuth()` is the consumer hook and throws if called outside its provider.

`frontened/src/context/ThemeContext.jsx`

Purpose: Manage UI light/dark preference.

- Reads the saved theme from `localStorage`, defaulting to light.
- Adds/removes the `dark` class on the HTML root element.
- Persists the selected value and exposes `theme` plus `toggleTheme()`.

`frontened/src/components/ProtectedRoute.jsx`

Purpose: Gate routes that require a signed-in user.

- Shows a spinner while session loading is unresolved.
- Renders children if `user` exists.
- Otherwise uses React Router `Navigate` to send the visitor to `/login`, preserving the attempted location.

`frontened/src/components/Shell.jsx`

Purpose: Shared authenticated application chrome.

- Renders the sticky header, mobile menu, sidebar navigation, user initials, theme toggle, sign-out action, page content, and footer.
- Uses `useAuth()` for the user and `signOut()`; after sign-out it navigates to login.
- Contains navigation to Dashboard, New Research, History, Sources & Library, Reports, and Settings.

`frontened/src/components/StatusBadge.jsx`

Purpose: Present job status consistently.

- Maps statuses such as pending, researching, completed, and failed to human-readable labels and visual styles.
- Used by dashboard cards, history rows, and report views.

`frontened/src/components/AuthLayout.jsx`

Purpose: Shared layout for login/signup pages.

- Provides consistent branding and visual structure for authentication screens.

`frontened/src/components/Footer.jsx`

Purpose: Shared footer.

- Renders product/footer content below pages supplied through `Shell`.

Frontend pages

`frontened/src/pages/Login.jsx`

Purpose: Email/password sign-in screen.

- Collects credentials and calls `signIn` from `AuthContext`.
- Shows Supabase errors to the user and navigates authenticated users into the app.

`frontened/src/pages/Signup.jsx`

Purpose: New-user registration screen.

- Collects full name, email, and password, then calls `signUp`.
- Passes the full name into Supabase user metadata.
- Handles both immediate sessions and email-confirmation flows returned by Supabase.

`frontened/src/pages/Dashboard.jsx`

Purpose: Primary research-brief composer and recent-job dashboard.

- `loadJobs()` calls `api.listResearch()` and shows only the current user's returned jobs.
- `handleSubmit()` validates the query, then navigates to `/research/new` with the query in router state.
- Displays example prompt buttons, current-user greeting, job status cards, and API errors.
- The actual research POST is deliberately owned by `ResearchProgress.jsx`, not this page.

`frontened/src/pages/ResearchProgress.jsx`

Purpose: Start research and display the waiting experience.

- Reads `location.state.query`; returns to Dashboard if no query was passed.
- Uses a ref to ensure only one `api.createResearch(query)` call occurs despite React effects/rerenders.
- Runs a timer across Planning, Researching, Validating, and Reporting UI stages while waiting for the HTTP response.
- On success, navigates to `/research/{job_id}`. On failure, displays an error/retry exit path.
- **Limitation:** its source/evidence/report statuses are visual estimates, not server events.

frontened/src/pages/ReportView.jsx

Purpose: Detailed view of one completed research job.

- Reads `jobId` from the route with `useParams()` .
- Loads the job first through `api.getJob(jobId)` .
- For completed jobs, concurrently loads the report, evidence, and sources.
- Parses `report.content_md` as JSON; despite its name, this DB field contains serialized report JSON rather than Markdown.
- Renders three tabs: **Report** sections/citations, **Evidence** claims/excerpts/confidence, and **Sources** external source links.
- Builds a `sourceById` map to connect each persisted evidence item to its persisted source.

frontened/src/pages/History.jsx

Purpose: Searchable archive of the user's research jobs.

- Loads jobs with `api.listResearch()` .
- Filters locally by brief text, formats dates, and navigates a clicked job to `ReportView` .

frontened/src/pages/Reports.jsx

Purpose: List and export completed reports.

- Loads all jobs, filters completed ones, then fetches each report.
- `parseReport()` parses stored JSON report content.
- `downloadReport()` creates a browser `Blob` and downloads a compact Markdown file.

frontened/src/pages/Sources.jsx

Purpose: Cross-job source/evidence library.

- For every returned job, concurrently fetches sources and evidence.
- Groups evidence by persisted `source_id` , attaches the job brief, and makes a searchable source card list.
- **Trade-off:** it can make many requests when a user has many jobs.

frontened/src/pages/Settings.jsx

Purpose: Settings screen.

- Provides user-facing settings-related UI. Authentication/session persistence remains controlled by Supabase and `AuthContext` .

frontened/src/pages/Methodology.jsx

Purpose: Explain the research methodology in the UI.

- Static/informational page describing the multi-stage research process.

`frontened/src/pages/AboutProject.jsx`

Purpose: Product/project information page.

- Static/informational route; it does not invoke the research backend.

4. Application layer — FastAPI files

`backened/backend/main.py`

Purpose: Build and expose the FastAPI application.

- Loads settings from `get_settings()` and initializes logging.
- Creates `FastAPI` with title, version, and description.
- Installs CORS middleware using configured origins; installs `RequestIDMiddleware`.
- Registers the `AppError` exception handler and all API routers.
- `GET /` returns service metadata; `GET /health` returns health status plus the request ID.

`backened/backend/core/config.py`

Purpose: Central environment configuration.

- `Settings(BaseSettings)` declares app name, environment, debug mode, Supabase URL/key, and comma-separated CORS origins.
- Loads `.env` using Pydantic Settings.
- `cors_origin_list` converts the configured comma-separated origins into a list.
- `get_settings()` is cached via `@lru_cache`, so settings are created once per process.

`backened/backend/core/auth.py`

Purpose: Server-side JWT/session verification.

- `get_current_user(authorization)` is a FastAPI dependency.
- Requires a correctly formatted `Bearer <token>` header.
- Calls `supabase.auth.get_user(token)` to validate the user with Supabase.
- Raises HTTP 401 for missing, malformed, invalid, or expired tokens; otherwise returns the Supabase user object.

`backened/backend/core/errors.py`

Purpose: Custom application error format.

- `AppError` carries an error name, machine code, detail, and HTTP status.
- `JobNotFound` and `ValidationFailed` are specialized error types.
- `app_error_handler()` converts these exceptions into consistent JSON error responses.

`backened/backend/core/logging.py`

Purpose: Logging setup.

- `configure_logging()` establishes the Python logging format/level used by routes, services, and AI pipeline modules.

backened/backend/middleware/request_id.py

Purpose: Request correlation.

- `RequestIDMiddleware.dispatch()` accepts an incoming `X-Request-ID` or generates a UUID.
- Places it on `request.state.request_id` for logs and sends it back as the response header.

FastAPI routes

backened/backend/api/research.py

Purpose: Main product API.

- `ResearchRequest` validates incoming JSON with a required `query`; `ResearchResponse` defines the POST response.
- `_ensure_owner(job, user)` raises 403 when a job belongs to another user.
- `GET /api/research/` lists only the caller's jobs.
- `POST /api/research/` validates the brief, creates a pending job, updates it to researching, runs `ResearchService.run_research`, marks completed, and returns title/summary.
- If POST fails it attempts to mark the job failed and returns HTTP 500.
- GET endpoints for one job, tasks, sources, evidence, validations, and report all authenticate, fetch the job, verify ownership, then query their repository.

backened/backend/api/reports.py

Purpose: Reserved reports router.

- Currently only exposes `GET /api/reports/test`; production report retrieval is presently under `/api/research/{job_id}/report`.

backened/backend/api/evidence.py

Purpose: Reserved evidence router.

- Currently only exposes `GET /api/evidence/test`; real evidence retrieval is under the research router.

backened/backend/api/feedback.py

Purpose: Reserved feedback router.

- Currently only exposes `GET /api/feedback/test`. The feedback table exists, but no create/read feedback workflow has been implemented.

Service and repositories

`backened/backend/services/research_service.py`

Purpose: Translate one pipeline result into persisted database records.

- `ResearchService.__init__()` supports dependency injection for the pipeline and repositories, useful for future tests.
- `run_research(query, job_id)` validates input, calls `ResearchPipeline.run(query)`, and logs duration/results.
- Writes tasks, sources, evidence, validations, and final report in that order.
- Builds `source_map` from temporary AI source IDs to database source rows, then `evidence_map` from temporary AI evidence IDs to database evidence rows.
- Those mappings ensure validation records use real persisted evidence UUIDs.

`backened/backend/repositories/research_job_repository.py`

Purpose: CRUD for `research_jobs`.

- `create_job(brief, created_by)` inserts a pending job.
- `list_jobs(created_by, limit)` orders newest-first and scopes the query by owner.
- `get_job(job_id)` fetches one row.
- `update_status(job_id, status)` changes state such as researching, completed, or failed.

`backened/backend/repositories/planner_task_repository.py`

Purpose: CRUD for `planner_tasks`.

- `create_task` saves task purpose as `task_type`, query, and queued status.
- `get_tasks` retrieves a job's task records; `update_status` is available for future per-task tracking.

`backened/backend/repositories/source_repository.py`

Purpose: CRUD for `sources`.

- `create_source` persists URL/title and optional fetch time; `get_sources` lists a job's source rows.
- Current persistence does not store all in-memory AI metadata such as publisher and source type.

`backened/backend/repositories/evidence_repository.py`

Purpose: CRUD for `evidence`.

- `create_evidence` saves claim, quote, relevance confidence, source UUID, job UUID, and initial pending status.
- `get_evidence` retrieves all evidence for a job.

`backened/backend/repositories/validation_repository.py`

Purpose: Persist/retrieve validation outcomes.

- `create_validation` turns boolean/scores/reason into a readable `notes` string and inserts an `ai_validation` record.
- `get_validations(evidence_id)` returns all validation rows for one evidence UUID.

`backened/backend/repositories/report_repository.py`

Purpose: Persist/retrieve reports.

- `create_report` serializes the Pydantic report dictionary as formatted JSON in `content_md`, despite the name implying Markdown.
- `get_reports(job_id)` returns newest-first report versions. The GET report route returns the first result.

`backened/backend/db/supabase_client.py`

Purpose: Create the privileged server-side Supabase client.

- Loads `SUPABASE_URL` and `SUPABASE_KEY` from environment.
- Fails immediately if either is missing, preventing partially configured runtime behavior.
- Exports one shared `supabase` client used by auth and repositories.

`backened/backend/db/seed.py`

Purpose: Seed/support database utility.

- Reserved for inserting development/seed data; it is not in the request-time research flow.

5. Intelligence layer — AI pipeline files

`backened/ai/pipeline/research_pipeline.py`

Purpose: Central seven-stage AI workflow coordinator.

- `ResearchPipeline.__init__()` constructs/injects `Planner`, `Researcher`, `Extractor`, `Validator`, `Reporter`, `CitationBuilder`, and `ReportLinker`.
- `_run_with_retry()` retries Gemini-related temporary 429/5xx failures, with exponential backoff or a retry delay parsed from provider errors.
- `run(query)` executes all stages in order, logs timing, checks every stage for empty results, and returns typed `ResearchResult`.
- Applies safety/cost caps: extracts from at most six sources, validates at most 30 evidence items, and limits report input.

`backened/ai/planner/planner_agent.py`

Purpose: Turn one user brief into research tasks.

- `PlannerAgent.create_plan(query)` prompts Gemini for an array of task ID/query/purpose objects.
- Removes possible Markdown code fences, parses JSON, confirms it is an array, and validates every item as `ResearchTask`.
- Raises a clear error if Gemini returns malformed JSON or invalid task objects.

`backened/ai/research/research_agent.py`

Purpose: Discover web sources for each task.

- `research(task)` sends the task query to `TavilySearchEngine.search`.
- Ignores results with missing URL/title and removes duplicate URLs within the task.
- Creates typed `Source` records with temporary IDs and retrieval timestamp.
- `_classify_source(url)` gives basic source classes for government, academic, Reuters, Bloomberg, or generic web domains.

`backened/ai/extraction/extraction_agent.py`

Purpose: Extract evidence claims from one source's page content.

- `extract(source)` asks Tavily to extract the source URL's content, trims it to 4,000 characters, then asks Gemini for up to three factual claims and direct excerpts.
- Validates required fields, clamps relevance score between 0 and 1, and produces `Evidence` objects.
- `_clean_response()` strips code fences and isolates a JSON array before parsing.
- Throws when no valid evidence is produced, contributing to the pipeline's fail-fast behavior.

`backened/ai/validation/validation_agent.py`

Purpose: LLM-based evidence quality assessment.

- `validate(evidences, sources)` builds a source map and validates evidence in batches of 15.
- `_validate_batch()` sends each claim/excerpt plus source metadata to Gemini and expects validity, credibility, recency, duplicate/conflict flags, and short rationale.
- Clamps scores to 0–1 and returns typed `ValidationResult` rows.
- This validates supplied material using an LLM; it is not a separate external fact-checking crawler.

`backened/ai/report/citation_builder.py`

Purpose: Deterministically turn sources into citations.

- `build(sources)` creates one `Citation` object per unique temporary source ID.
- No Gemini call is made here; it preserves source URL/title/publisher/date fields.

`backened/ai/report/report_agent.py`

Purpose: Produce the final market-research report.

- `generate_report(tasks, evidences, validations, citations)` maps validations to evidence and excludes explicitly invalid evidence.
- Ranks evidence by credibility and recency and sends the strongest 15 to Gemini.
- Requires Gemini to return structured JSON with title, summary, findings, signals, competitor observations, implications, recommendations, and evidence IDs.
- Validates output as `Report`, then attaches citations. `_clean_response()` isolates the JSON object.

`backened/ai/report/report_linker.py`

Purpose: Create evidence-to-source traceability objects.

- `_build_maps()` maps evidence ID to evidence and source ID to citation.
- `link_item()` resolves every report item's evidence IDs into unique linked source title/URL/publisher records.
- `link_report()` applies the same linking to all report sections and returns a `LinkedReport`.
- The resulting linked report currently remains in memory inside `ResearchResult`; it is not separately saved or served to the current frontend.

LLM and browser adapters

`backened/ai/llm/base.py`

Purpose: LLM abstraction.

- Defines the abstract LLM contract, allowing agents to depend on `generate(prompt)` rather than a specific provider implementation.

backened/ai/llm/gemini.py

Purpose: Shared Google Gemini provider implementation.

- Loads `GOOGLE_API_KEY` and fails fast if absent.
- `GeminiLLM.__init__()` creates a `google.genai.Client` and configures primary/fallback model names.
- `_chat_generate()` creates a Gemini chat and sends one prompt.
- `_generate_with_retry()` retries transient quota/service failures per model.
- `generate()` tries primary then fallbacks and translates provider errors into application-friendly runtime errors.

backened/ai/browser/search.py

Purpose: Search abstraction.

- Defines abstract `SearchEngine.search(query)` , enabling a real Tavily implementation or test/mock replacement.

backened/ai/browser/tavily_search.py

Purpose: Tavily web research adapter.

- Loads `TAVILY_API_KEY` and creates `TavilyClient` .
- `search(query)` calls Tavily basic search and requests two results per research task.
- `extract(urls)` requests raw content for URLs used by the extraction agent.

backened/ai/browser/mock_search.py

Purpose: Test/development search substitute.

- Provides controlled search-like responses without contacting Tavily, useful when writing agent/pipeline tests or developing offline.

Typed schema files

These files use Pydantic models. They validate LLM output at each boundary, which prevents malformed JSON from silently moving into later stages.

File	Model / working
ai/schemas/research_task.py	<code>ResearchTask</code> : task ID, targeted query, and purpose from the planner.
ai/schemas/source.py	<code>Source</code> : temporary source ID, URL, title, type, publisher/date metadata, and retrieval time.
ai/schemas/evidence.py	<code>Evidence</code> : claim, source excerpt, entity, topic, relevance score, source ID.
ai/schemas/validation.py	<code>ValidationResult</code> : evidence ID, validity/duplicate/conflict flags, credibility/recency scores, and reason.
ai/schemas/citation.py	<code>Citation</code> : citation ID plus source identity and bibliographic URL/title metadata.
ai/schemas/report_item.py	<code>ReportItem</code> : one report sentence/text plus its supporting evidence ID list.

ai/schemas/report.py	Report : final structured strategy report and citations.
ai/schemas/linked_report.py	LinkedReport , LinkedReportItem , and LinkedSource : report sections enhanced with resolved sources.
ai/schemas/research_result.py	ResearchResult : complete in-memory return object containing report, linked report, tasks, sources, evidence, validations.

6. Data layer — migrations and database files

The PostgreSQL database is hosted by Supabase. Run migration files in numeric order. Tables use UUID primary keys; a job's `created_by` references Supabase `auth.users` so API ownership checks can isolate each user's work.

File	Working / database responsibility
db/migrations/001_initial_schema.sql	Initial extension, core tables, foreign keys, permitted job statuses, and base indexes. It creates research jobs, tasks, sources, evidence, validations, memory, reports, and feedback foundations.
db/migrations/002_planner_tasks.sql	Idempotent creation of <code>planner_tasks</code> , which stores the decomposed research plan associated with a job.
db/migrations/003_sources.sql	Idempotent creation of <code>sources</code> ; each source belongs to a research job and may later support many evidence items.
db/migrations/004_evidence.sql	Idempotent creation of <code>evidence</code> ; stores claim, quote, confidence and source/job relationships.
db/migrations/005_validation_records.sql	Idempotent creation of validation records linked to evidence.
db/migrations/006_memory.sql	Enables pgvector and creates <code>memory_records</code> with 1536-dimensional embeddings for future semantic retrieval. Current runtime code does not yet populate/query it.
db/migrations/007_reports.sql	Idempotent creation of versioned reports linked to jobs.
db/migrations/008_feedback.sql	Idempotent creation of reviewer feedback records linked to reports and Supabase users.
db/migrations/009_indexes.sql	Creates indexes for evidence-by-job, tasks-by-job, and pgvector cosine similarity queries.

7. Supporting/configuration/documentation files

File	Working
frontened/package.json	Frontend dependency manifest and scripts: <code>dev</code> , <code>build</code> , <code>lint</code> , <code>preview</code> .
frontened/vite.config.js	Vite development/build configuration, including React and Tailwind integration.
frontened/vercel.json	Vercel deployment/routing configuration for the SPA.
frontened/.env.example	Template for browser-safe Supabase values and API base URL.
backened/backend/requirements.txt	Python dependencies: FastAPI, Uvicorn, Pydantic Settings, Supabase, dotenv, Google GenAI, Tavily.
backened/.env.example and root .env.example	Templates for server credentials. Do not commit filled secret files.

README.md	Top-level architecture, setup, deployment, and product documentation.
project-docs/ARCHITECTURE.md	High-level architecture and seven-agent workflow explanation.
project-docs/API.md	Endpoint overview and authentication expectations.
project-docs/DEPLOYMENT.md	Vercel frontend, Render backend, and Supabase deployment guidance.
project-docs/EVALUATION.md	Evaluation-related project documentation.
__init__.py files	Python package markers. They make directories importable packages; most contain no application behavior.

8. Full request lifecycle with files involved

1. **Login:** `Login.jsx` → `AuthContext.jsx` → `supabaseClient.js` → Supabase Auth.
2. **Brief entry:** `Dashboard.jsx` collects the query and navigates to `ResearchProgress.jsx`.
3. **Authenticated API call:** `ResearchProgress.jsx` → `api/client.js` adds the access token → `api/research.py:create_research()`.
4. **Authorization and job:** `core/auth.py:get_current_user()` verifies the JWT; `research_job_repository.py:create_job()` stores the owner-scoped pending job.
5. **Pipeline:** `research_service.py:run_research()` → `research_pipeline.py:run()` → planner, researcher, extractor, validator, citation builder, report agent, linker.
6. **External providers:** `gemini.py` sends reasoning prompts; `tavily_search.py` searches and extracts web content.
7. **Persistence:** `ResearchService` writes output through task/source/evidence/validation/report repositories into Supabase tables.
8. **Viewing:** `ReportView.jsx` calls job/report/evidence/source endpoints through `client.js`, parses the stored report JSON, and renders it.

9. Current implementation notes for developers

- The correct source folders are named `frontened` and `backened` in this repository; changing those names requires updating imports/deployment commands.
- Research is synchronous and may exceed typical serverless/proxy request limits for complex queries. A production upgrade should use a background queue and server-reported job stages.
- The report linker creates rich traceability data but that `LinkedReport` is not currently persisted or returned by the API.
- Validation data is stored as a readable notes string. Structured validation columns would make analytics and filtering easier.
- The sources UI makes per-job source/evidence calls. A consolidated server endpoint would improve speed for users with extensive history.
- Provider keys and the backend Supabase key belong in environment variables only. Never expose them in frontend code or commit an active `.env` file.